

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices

Arjit Tripathi

arjit.tripathi2023@vitstudent.ac.in

Medha Sriram

medha.sriram2023@vitstudent.ac.in

Heer Shah

shah.heer2023@vitstudent.ac.in

Dr. Shalini L

l.shalini@vit.ac.in

Abstract—On-device language models frequently require situational context—physical location, proximate companions, time of day—prepended to user prompts to generate appropriate responses. Ingesting this context on mobile hardware is computationally expensive. On a OnePlus 11R running Qwen2.5-0.5B-Instruct (Q4_K_M), prefilling a 439-token context prompt averaged 14.9 s. Server systems cache key-value (KV) states across queries to avoid recomputation, but that mechanism relies on concurrent request streams from multiple users that do not exist on a personal smartphone. A smartphone does possess ambient sensors. CAP-KVC uses GPS, Bluetooth Low Energy (BLE) proximity, and time-of-day readings to rank a small set of pre-authored situational contexts, prefills likely candidates during device idle periods, and retains up to three resident llama.cpp sequence states. When an interaction occurs, the engine decodes only the user’s intent tokens. Across 30 interleaved trials per condition in randomized order, restoring a 44-token context and decoding 9 intent tokens averaged 366 ms, compared with 1847 ms for the matched 52-token inline prefill (a $5.0\times$ reduction; Welch $t(32.3) = 15.5, p < 10^{-15}$). State restoration added 2.9 ms. Over 100 simulated days spanning 6,400 interactions across five contexts, sensor-driven residency selected the correct context for 98.5% of interactions (mean 98.3%, SD 0.16 pp across 20 seeds) versus 92.2% for request-driven LRU caching, trading first-visit cold misses for 1.5% wrong-context selections. A deterministic Markov baseline reached 99.7%, reflecting the predictability of synthetic schedules. Undetected GPS spoofing increased wrong-context selections to 38.9%. The prototype is evaluated on one phone, one model, five contexts, and synthetic traces.

Index Terms—Key-value cache, edge inference, predictive caching, language models, mobile systems, sensor-driven context.

I. INTRODUCTION

When a smartphone runs an on-device language model to assist a user, generating an appropriate response often requires situational context: the user’s physical location, nearby companions, and the time of day. In augmentative and alternative communication (AAC) [1], for example, an individual may tap a short prompt like “need water.” Expanding that brief input into a contextually appropriate utterance depends on the environment. A hospital ward at mid-morning calls for different phrasing than home in the evening with family present. The conventional method prepends a descriptive context prompt to the user’s intent tokens and processes the entire sequence through the model.

Ingesting these context prompts on mobile processors incurs substantial latency before generation begins. On an Android smartphone (OnePlus 11R) running Qwen2.5-0.5B-Instruct

quantized to Q4_K_M, evaluating a 439-token context prompt required an average of 14.9 s (Section V-B). Even a compact 52-token prompt required 1.8 s, corresponding to roughly 34 ms per prompt token. In conversational turn-taking, an initial delay of nearly two seconds before speech output begins disrupts natural communication.

Server-based serving engines avoid re-evaluating prompt prefixes by retaining intermediate key-value (KV) attention states across requests. Radix-tree caching in SGLang [6], paged KV sharing in vLLM [7], modular attention reuse in Prompt Cache [8], and retrieved-passage caching in RAG-Cache [9] exploit query overlap across thousands of users. One user’s request populates the cache; subsequent users matching that prefix skip the prefill stage. A personal smartphone lacks multi-user request concurrency. The query that would populate a reactive cache is the user’s first turn in a newly entered environment, forcing that interaction to bear the full prefill delay.

While a smartphone receives no external query stream, it continuously gathers ambient sensor data. We exploit this observation in CAP-KVC (Context-Aware Predictive Key-Value Cache priming). CAP-KVC uses GPS coordinates, BLE proximity to beacon anchors, and the device clock to score and rank a small library of pre-written situational contexts. During idle periods, the application prefills high-ranking contexts into native llama.cpp sequence slots and saves their KV states to private storage, maintaining up to three resident states in memory. When the user interacts via button tap or gaze dwell, the engine resumes the active resident slot and decodes only the intent tokens. On the OnePlus 11R, restoring an existing 44-token context and decoding 9 intent tokens averaged 366 ms, compared with 1847 ms for the matched 52-token inline prefill (a $5.0\times$ reduction, $n = 30$ each; Welch $t(32.3) = 15.5, p < 10^{-15}$). Deserializing the cached state from storage added an average overhead of 2.9 ms (median 2.1 ms).

We evaluated context selection across 100 simulated days containing 6,400 user interactions across five daily contexts. Sensor-driven residency selected the correct context for 98.5% of interactions, compared with 92.2% for a request-driven LRU cache that misses on every context transition. The sensor approach trades cold misses for a 1.5% rate of wrong-context selections. This evaluation is subject to specific constraints: one physical phone, one model architecture, five candidate contexts, and synthetic traces.

TABLE I
ARCHITECTURAL COMPARISON: REQUEST-DRIVEN KV REUSE VS. CAP-KVC.

Property	Request-driven reuse	CAP-KVC
Environment	Multi-tenant server	Single user on mobile device
Cache trigger	Upon user query arrival	Ambient sensing during idle
Context signal	Query token prefix	Multi-modal (GPS, BLE, time)
First turn in context	Full prefill latency	Restored from primed slot
Primary failure mode	Cold miss	Wrong context or cold miss

This paper makes three contributions:

- 1) A sensor-driven predictive KV caching architecture for single-user mobile inference: deterministic multi-modal context scoring from GPS, BLE, and clock inputs, bounded slot residency governed by a hysteresis threshold, and a two-tier fallback mechanism.
- 2) An Android implementation built on llama.cpp via JNI, featuring sequence-slot isolation, rollback of user intent tokens so primed contexts remain reusable, atomic disk serialization, and location validation filters.
- 3) An evaluation characterizing matched-context prefill latency through randomized interleaved benchmarking, prediction accuracy across noisy and jittered synthetic traces, baselines including an empirical Markov ceiling, and resilience against forged sensor inputs.

II. RELATED WORK

Request-driven KV reuse. Serving frameworks for large language models amortize prompt computation by caching KV tensors in host or accelerator memory. SGLang [6] indexes KV tensors using a radix tree over token prefixes; vLLM [7] allocates KV memory in fixed-size pages to support prefix sharing; Prompt Cache [8] and CacheBlend [10] precompute and fuse attention states for modular prompt components; CacheGen [11] compresses KV tensors for streaming across servers; RAGCache [9] structures cache replacement around document retrieval patterns. These frameworks operate reactively: a cache entry is populated only after a request containing those specific tokens arrives at the server. Cache utility depends on multi-user query concurrency. On a single smartphone, no concurrent query stream exists. The initial query in a newly visited context must execute a full prefill before any state can be cached (Table I). CAP-KVC alters the trigger and timing of cache population: it prefills KV states proactively during idle time based on physical sensor readings, before any interaction begins.

KV cache compression. A separate research direction aims to reduce the memory footprint of attention states during inference. Heavy-hitter pruning in H₂O [12] and Scissorhands [13], attention-sink retention in StreamingLLM [15], key-value reduction in SnapKV [14], and low-precision representations in KVQuant [16] reduce storage within an active sequence.

These methods address intra-sequence memory consumption and bandwidth. They do not eliminate the prefill computation needed to read an initial prompt. Compression techniques are orthogonal to our design: a compressed KV tensor representation can still be primed and restored using the proactive caching mechanisms described in this work.

On-device inference runtimes. Post-training quantization [4], [5], mobile-optimized model architectures like MobileLLM [17], embedded runtime engines such as llama.cpp [2] and MLC-LLM [18], flash-memory parameter offloading [19], and heterogeneous CPU/NPU execution [20] enable sub-billion-parameter language models to execute on mobile silicon. These runtimes accelerate per-token decode throughput. However, prefilling several dozen or hundred context tokens before the first token can be decoded remains a major latency bottleneck on mobile CPUs. CAP-KVC builds on llama.cpp to remove this prefill overhead from the interactive path.

III. SYSTEM DESIGN

The system architecture separates background context preparation from foreground interaction (Fig. 1). A background service sweeps onboard sensors periodically, scores each candidate context, and updates the set of resident KV states. When the user interacts, the foreground service selects the highest-ranking resident state and decodes only the intent.

A. Data Management

The application stores a fixed catalog of $M = 5$ pre-written situational contexts: home morning, home evening, hospital ward, therapy room, and caregiver visit. Each context c_i defines a system prompt p_i averaging 44 tokens (specifying persona, spatial environment, and vocabulary guidelines). The system also maintains serialized KV cache dumps (‘.kv’ files) in app-private internal storage, ensuring persisted states can be loaded across process restarts.

B. Context Scoring

Each context c_i defines an expected time anchor t_i , a reference GPS coordinate ℓ_i , and a list of expected BLE beacon identifiers A_i with importance weights w_{ij} . When the background service gathers sensor inputs, it computes a score for each context:

$$S(c_i) = \alpha S_{\text{time}} + \beta S_{\text{gps}} + \gamma S_{\text{ble}}, \quad \alpha + \beta + \gamma = 1, \quad (1)$$

where each modal score evaluates a specific sensor dimension:

$$S_{\text{time}} = \exp(-d_{\text{circ}}(t, t_i)^2 / 2\sigma^2), \quad \sigma = 2 \text{ h}, \quad (2)$$

$$S_{\text{gps}} = \exp(-d_{\text{hav}}(\ell, \ell_i) / \lambda), \quad \lambda = 0.1 \text{ km}, \quad (3)$$

$$S_{\text{ble}} = \sum_{j \in A_i} w_{ij} \mathbb{I}[j \text{ seen}] / \sum_{j \in A_i} w_{ij}. \quad (4)$$

The circular distance d_{circ} wraps around midnight (treating 11:55 p.m. and 12:05 a.m. as 10 minutes apart). The spatial score S_{gps} applies the Haversine distance d_{hav} , decaying to 0.37 at a displacement of 100 m. The BLE score S_{ble} measures the detection ratio of expected beacon anchors.

Background: sweep every 60 s; BLE-free check every 15 min

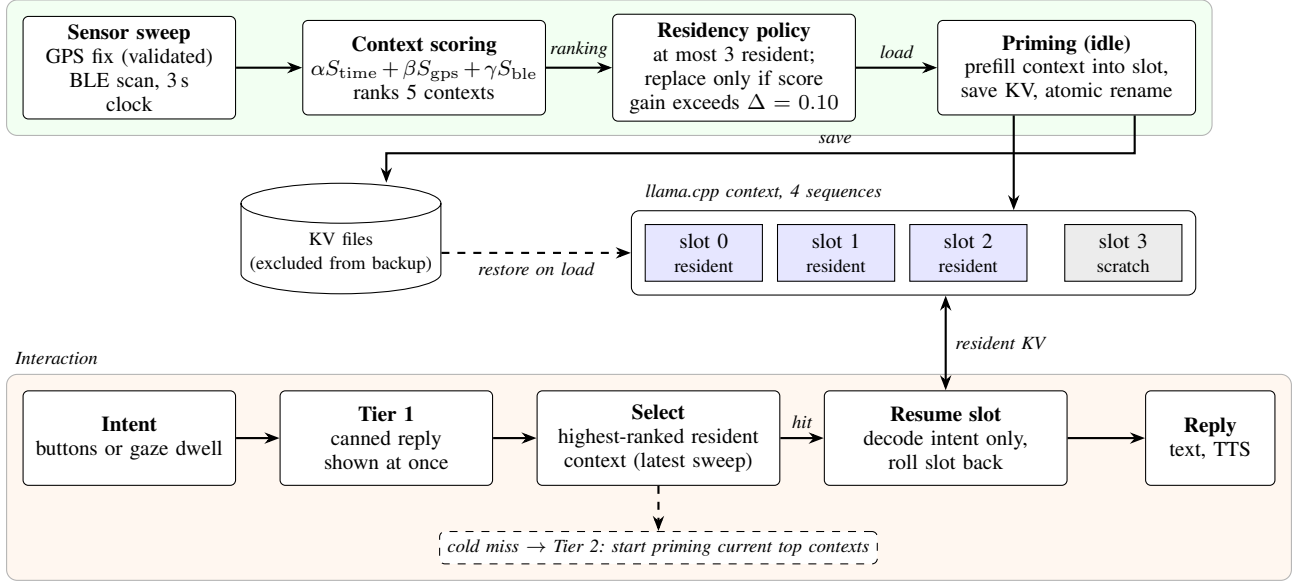


Fig. 1. CAP-KVC architecture. A background sweep evaluates candidate contexts from GPS, BLE, and time inputs, maintains up to three primed KV states across llama.cpp sequence slots, and persists each state to internal storage. When an interaction begins, Tier 1 immediately presents a canned response, then the engine resumes the highest-ranked resident slot to decode the user’s intent. If no matching state is resident, background priming is scheduled (Tier 2). Slot 3 handles inline inference and benchmark executions without evicting resident states.

The combination weights α , β , and γ derive from dynamic sensor quality indicators rather than learned parameters. Time receives a constant weight $w_t = 0.4$. GPS accuracy a (in meters) scales the location weight as $w_g = \exp(-a/50 \text{ m})$, falling to zero if no location fix is returned. BLE confidence w_b averages the logistic mapping $1/(1 + e^{-0.1(r+70)})$ over detected beacon RSSI values r , setting $w_b = 0$ if no beacons appear. Normalizing these values yields α, β, γ . Contexts whose aggregate score falls below 0.01 are discarded. In this paper, “predictive” refers to proactively prefilling states prior to interaction rather than executing a statistical machine learning classifier.

C. Residency Management and Hysteresis

Device RAM constrains resident capacity to $K = 3$ contexts out of $M = 5$. The llama.cpp context reserves four sequence slots: three for resident states and one scratch slot.

Following each sweep, the router sorts all candidates by score. Contexts that are already resident retain their slots. If an unprimed context qualifies for residency and an unoccupied slot exists, it is loaded into that slot. When all three slots are occupied, a new candidate evicts the lowest-ranking resident only if its score exceeds the incumbent’s score by more than a hysteresis threshold Δ :

$$S(c_{\text{new}}) > S(c_{\text{resident_min}}) + \Delta. \quad (5)$$

Without this hysteresis margin, minor GPS jitter or intermittent BLE packet drops cause repeated cache evictions between consecutive sweeps. The parameter K controls residency capacity; Δ is an independent score difference threshold. The deployed

prototype sets $\Delta = 0.10$. Section V-D analyzes the impact of varying this threshold.

D. Priming, Interaction, and Fallback

Priming executes during idle time. The engine decodes the 44-token prompt into an assigned sequence slot, writes the resulting KV tensors to a temporary file on disk, and renames the file atomically. This guarantees that process crashes do not leave corrupted or partially written cache files. A background WorkManager task wakes every 15 minutes to evaluate coarse location and time without running the BLE radio, ensuring that top contexts remain primed without excessive battery draw.

When the user provides an intent input, the app checks the resident slots for the highest-scoring candidate. If a matching resident state is available, the engine appends the user’s intent tokens q to that slot, generates the response, and then rolls the slot back by discarding tokens added during generation. This rollback preserves the primed context state for subsequent turns.

If the router’s top-ranked context is not resident in memory (a cold miss), a two-tier fallback responds:

- **Tier 1:** A canned acknowledgment is rendered immediately on screen and passed to the text-to-speech engine.
- **Tier 2:** The system schedules a background prefill of the missing context into an available sequence slot.

If sensor noise or spoofing causes the system to select an incorrect resident context, the model decodes the user’s intent against that wrong template. Because the candidate pool is restricted to five vetted clinical contexts, the resulting text remains bounded to the application’s domain rather than generating unconstrained output.

IV. IMPLEMENTATION

AACBridge is implemented as an Android application (Kotlin, targetSdk 34, minSdk 26, compiled for arm64-v8a). Inference is handled by a native C++ module embedding llama.cpp [2]. The target model is Qwen2.5-0.5B-Instruct [3] in Q4_K_M GGUF format, chosen as a compact instruction-tuned model compatible with mobile memory limits.

JNI Interface. The native boundary in llama_jni.cpp exports 14 native functions to Kotlin: three lifecycle operations (initializeBackend, initializeModel, release); four KV cache operations (saveKVCache, loadKVCache, clearKVCache, resetSlot); three inference functions (runInference, prefillOnly, resumeInference); and four telemetry getters (getLastPrefillMs, getLastGenMs, getLastPromptTokens, getLastGenTokens). All native calls pass through a single ReentrantLock in Kotlin because internal llama.cpp execution contexts are not thread-safe.

Sequence Allocation. The llama.cpp context is initialized with four sequence slots of 1024 tokens each. Slots 0, 1, and 2 hold resident context states; slot 3 serves as an isolated scratch buffer for inline inference and benchmarking, preventing inline executions from invalidating resident KV states. Qwen2.5-0.5B contains 24 transformer layers with 2 KV heads of dimension 64. Using 16-bit floating point storage, KV data consumes 12 KiB per token, totaling 48 MiB allocated across the four sequence slots at initialization. Prior to this sequence slot restructuring, a system memory snapshot recorded 627 MB proportional set size (PSS), including 151 MB native heap and 391 MB memory-mapped model weights; memory was not re-measured after moving to the four-sequence slot layout.

Sensor Processing. A foreground service executes the sensor sweep every 60 s, performing a 3-second BLE scan via BluetoothLeScanner and querying FusedLocationProviderClient. The implementation validates location fixes before acceptance: fixes flagged as mock by Android (isMock() or isFromMockProvider()) are rejected, fixes older than 120 s are discarded, and reported accuracy must fall between 2 m and 500 m. If cached fixes fail validation, a fresh network or GPS fix is requested. An auxiliary WorkManager job executes every 15 minutes without BLE.

Intent Input and Security. The user interface supports touch buttons and gaze-dwell input. Application backup is disabled (android:allowBackup="false"), and data_extraction_rules.xml excludes the KV cache directory from device-to-device transfers and cloud storage.

V. EVALUATION

A. Experimental Setup

All latency benchmarks were performed on a OnePlus 11R smartphone (Qualcomm Snapdragon 8+ Gen 1, 8 GB

TABLE II
LATENCY COMPONENTS (MS), MEDIAN [IQR], $n = 30$ EACH, INTERLEAVED RANDOMIZED TRIAL ORDER. CAP-KVC RESTORES A 44-TOKEN CONTEXT AND DECODES 9 INTENT TOKENS.

Condition (tokens)	Prefill	Generation	Restore
No context (9)	292 [265, 388]	4740 [3319, 6718]	–
Inline, matched (52)	1741 [1363, 2223]	4857 [3319, 7837]	–
Inline (89)	2368 [2308, 3546]	4289 [3360, 6127]	–
Inline (206)	7104 [5458, 7963]	5112 [3771, 6465]	–
Inline (439)	13989 [12160, 17142]	5677 [4311, 7188]	–
CAP-KVC (9)	293 [267, 467]	3922 [3531, 5773]	2.1 [1.1, 4.0]

LPDDR5 RAM, Android 14). The llama.cpp engine was configured with 4 compute threads, batch size 512, greedy decoding, and a cap of 64 generated tokens. The benchmark used a debug build.

To mitigate thermal throttling bias and operating system background scheduling noise, measurements followed an interleaved protocol. The benchmark executed 30 rounds; each round evaluated all six test conditions in a randomized sequence governed by seed 20261005, preceded by 2 discarded warmup rounds. Gaze input was isolated during testing by setting an isBenchmarkActive guard: 336 gaze events were intercepted and blocked, with zero leakage into the inference pipeline. Every trial logged device telemetry: thermal-zone temperature ranged between 65.8°C and 78.0°C (median 69.7°C); big-core (cpu7) frequency ranged between 787 MHz and 1402 MHz, operating at its 1.4 GHz maximum in 68% of trials and throttling below 1000 MHz in 11%; battery temperature returned −1.0 (unsupported on this build without system privileges). Prefill and generation durations were measured using std::chrono timestamps surrounding llama_decode; restore time was recorded around the cache load operation. All 180 trial logs are preserved in the repository.

B. Matched-Context Latency

We measured the execution latency removed when a primed context is restored rather than decoded inline from raw text.

In the CAP-KVC condition, a 44-token context prompt resides in memory; the model decodes only the 9 intent tokens. In the matched inline condition, the identical 44-token context and 9 intent tokens are processed together as a single 52-token prompt. Additional inline conditions (89, 206, and 439 tokens) illustrate scaling behavior as context size expands.

Table II summarizes the latency breakdown. The overhead of loading a serialized KV state from internal storage averaged 2.9 ms (median 2.1 ms). Prefilling the 9 intent tokens in CAP-KVC averaged 366 ms (SD 122 ms; median 293 ms). In contrast, the matched 52-token inline prefill averaged 1847 ms (SD 510 ms; median 1741 ms). This represents a $5.0\times$ reduction in prefill time (Welch $t(32.3) = 15.5$, $p < 10^{-15}$). Estimated time to first token (restore plus prefill plus an initial generation step) dropped from approximately 1.8 s inline to 0.37 s with CAP-KVC.

Inline prefill latency scaled at an empirical rate of 34.1 ms per prompt token (ordinary least squares, $R^2 = 0.88$; Fig. 2).

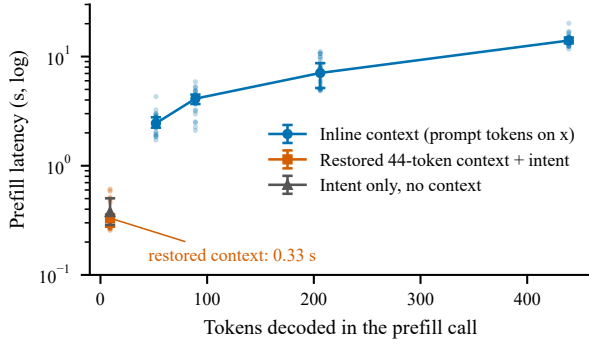


Fig. 2. Prefill latency versus prompt token count (log scale). Points represent individual benchmark trials; markers denote median and IQR ($n = 30$ per condition).

At 439 tokens, inline prefill reached an average of 14,941 ms (14.9 s; median 13,989 ms), which is $40.8\times$ longer than the CAP-KVC 9-token prefill. This $40.8\times$ figure reflects the latency avoided when handling longer prompts rather than a matched speedup. Restore latency was benchmarked solely for the 44-token context state; scaling across variable cache sizes was not evaluated.

Autoregressive token generation dominated overall interaction time, spanning a median of 3.9 s to 5.7 s across conditions when generating up to 64 tokens. Consequently, saving 1.5 s during prefill does not produce a $5.0\times$ reduction in total end-to-end response time; the benefit is isolated to time to first token. Trial position within an interleaved round showed low correlation with prefill latency (Spearman $|\rho| \leq 0.13$ across conditions), confirming that the randomized interleaved protocol successfully controlled for ordering bias. Thermal-zone temperature exhibited moderate correlation with prefill ($|\rho| \leq 0.30$).

C. Prediction and Cache Policy

Traces. Because multi-day physical sensor traces from individuals using AAC were unavailable, we generated synthetic traces (seed 42). The simulation models 100 days following a five-context routine between 06:00 and 22:00, with context transition times perturbed daily by uniform jitter $U(\pm 0.5 \text{ h})$. A sensor sweep occurs every minute; an interaction is generated every 15 minutes (totaling 6,400 interactions). Baseline noise parameters include: Gaussian GPS displacement $\mathcal{N}(0, 50 \text{ m})$, 10% GPS fix dropouts, reported GPS accuracy $\mathcal{N}(15, 10) \text{ m}$, 80% per-beacon detection with RSSI $\mathcal{N}(-65, 10) \text{ dBm}$, 15% BLE scan failure, and 10% spurious beacon sightings. A high-noise scenario applies $\pm 1.5 \text{ h}$ transition jitter, 200 m GPS displacement, 30% GPS dropouts, 50 m reported accuracy, 50% BLE detection, and 40% BLE scan outages. Because the context time anchors align with the simulated schedule, temporal evidence is structurally favorable.

Simulation. The trace harness executes the production Kotlin router and residency policy. Priming delay is not simulated: contexts scheduled for residency at one sweep are assumed available by the subsequent simulated minute.

TABLE III
INTERACTION OUTCOMES ACROSS 6,400 SYNTHETIC INTERACTIONS (%) AND CACHE OPERATIONS PER DAY. $\Delta = 0.10$ UNLESS SPECIFIED.

Policy / scenario	Correct	Wrong	Cold	Loads/d
Oracle (always resident)	100.0	0.0	0.0	5.0
Request-driven LRU, no sensing	92.2	0.0	7.8	5.0
Random ranking	20.7	79.3	0.0	1011.0
Time only	92.2	7.8	0.0	5.0
GPS only	87.9	12.1	0.0	45.7
BLE only	92.6	6.9	0.5	115.8
All sensors (deployed)	98.5	1.5	0.0	31.6
All sensors, $\Delta = 0.00$	98.5	1.5	0.0	65.9
GPS 70% unavailable	98.2	1.8	0.0	116.7
High noise stress	90.6	9.4	0.0	106.2
GPS forged, mock flag ignored	61.1	38.9	0.0	18.2
GPS forged, mock flag rejected	98.3	1.7	0.0	75.8
Cloned BLE beacons	90.1	9.9	0.0	103.2

Metrics. For each interaction, let c^* denote the true environment context and $\hat{c}$ the context selected by the system. Interactions are partitioned into three mutually exclusive outcomes: *correct* ($\hat{c} = c^*$); *wrong context* ($\hat{c} \neq c^*$, where a resident context was used but did not match ground truth); and *cold miss* (no candidate context was resident in memory). *Loads per day* measures how many times a context was primed or restored into a resident slot, reflecting background compute activity.

Baselines. We evaluated four comparison baselines: *Oracle* (c^* always resident, theoretical upper bound); *request-driven LRU* (sensing disabled; true context revealed only upon interaction, cached in a 3-slot LRU cache); *random ranking* (uniform random scores per sweep under the deployed residency rule); and *single-sensor ablations* (time-only scoring; GPS-only and BLE-only scoring with $w_t = 0.001$ to prevent temporal leakage through score normalization).

With all sensors active, the deployed configuration selected the correct context for 98.5% of interactions (Table III). Request-driven LRU reached 92.2% correct and incurred zero wrong-context errors, since it only caches verified requests; its 7.8% failure rate consisted entirely of cold misses on first visits. Sensor-driven priming eliminated first-visit cold misses at the cost of 1.5% wrong-context selections.

GPS alone could not differentiate the three contexts situated at the home location (morning, evening, caregiver visit), yielding 87.9% accuracy. BLE alone suffered cold misses in 0.5% of interactions where no registered beacon was detected. Time alone reached 92.2%, though this high figure is an artifact of synthetic schedule construction.

D. Hysteresis Margin Sweep

Fig. 3 evaluates the effect of varying the hysteresis margin $\Delta \in [0.00, 0.50]$. Increasing Δ from 0.00 to the deployed 0.10 reduced loads per day from 65.9 to 31.6 under normal noise, and from 297.5 to 106.2 under high noise. Context accuracy remained unchanged at 98.5% (normal) and 90.6% (high noise) across margins up to 0.50, demonstrating that genuine context transitions generate score shifts larger than

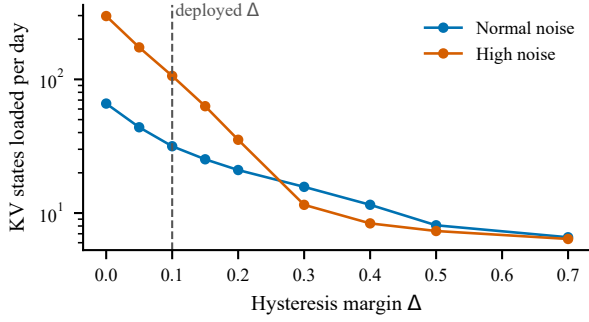


Fig. 3. Cache loads per day versus hysteresis margin Δ (100 synthetic days). Correct selection remained constant at 98.5% (normal) and 90.6% (high noise) across all margins up to 0.5.

these margins. While larger margins would reduce loads further on these synthetic traces, we maintain $\Delta = 0.10$ to preserve sensitivity to subtle transitions on real-world inputs.

E. Markov Successor Baseline

Because the synthetic routine visits five contexts in a fixed sequence, transitions are deterministic: `home_morning` is consistently followed by `hospital_ward`. We trained a first-order Markov model $P(c_{t+1} | c_t)$ on the first 50 days and evaluated it on the final 50 days. The Markov predictor attained 99.7% accuracy with 6.0 loads per day, exceeding the full sensor router (98.5% correct, 31.6 loads/day).

This result demonstrates that the synthetic benchmark schedule is trivially predictable; the Markov baseline exploits this fixed structure without requiring sensor inputs. On irregular real-world schedules containing skipped, delayed, or out-of-order visits, Markov transition probabilities would degrade. We report this baseline to make the predictability ceiling explicit.

F. Residency Capacity (K) Ablation

We varied resident slot capacity across $K \in \{1, 2, 3\}$. With $K = 1$, the system maintains a single resident state, switching at every transition: accuracy reached 99.4% (0.6% wrong context) with 10.7 loads per day. Because only one context is held, there is no competition from stale resident states. However, every context transition requires a prefill or restore operation. With $K = 2$, accuracy was 98.5% (1.5% wrong context) with 39.1 loads per day. With $K = 3$ (deployed), accuracy remained 98.5% while loads per day decreased to 31.6, as incoming contexts frequently already occupied a resident slot.

G. Multi-Seed Robustness

To examine sensitivity to random noise generation, we evaluated the full router across 20 independent random seeds (each spanning 100 days under standard noise). Correct context selection averaged 98.3% (SD 0.16 pp), ranging from 98.03% (seed 8) to 98.58% (seed 4). LRU accuracy remained constant at 92.19% across all seeds due to the deterministic schedule. The performance advantage of sensor-driven residency over LRU remained steady at +6.1 pp across seeds.

H. Schedule Variation

We tested robustness against irregular schedules using traces with ± 1.0 h boundary jitter (5 seeds), ± 2.0 h jitter, doubled interaction rates (7-minute intervals), and combined high jitter (± 1.5 h jitter with 150 m GPS noise). Mean accuracy was 97.7% (± 1.0 h), 95.6% (± 2.0 h), 98.3% (variable dwell), and 95.9% (combined high jitter). As temporal boundaries become ambiguous under ± 2.0 h shifts, wrong-context selections rose to 4.4% (compared with 1.5% under ± 0.5 h jitter).

I. Abstention and Risk-Coverage

When the score margin between the top two candidate contexts is narrow, the router can abstain from selecting a resident context and trigger Tier-1 canned responses. Sweeping an abstention margin threshold τ from 0.00 to 0.20 showed: at $\tau = 0.00$ (no abstention), 98.5% correct, 1.5% wrong context, 0.0% abstention; at $\tau = 0.05$, 97.1% correct, 0.8% wrong context, 2.1% abstention; at $\tau = 0.10$, 94.6% correct, 0.4% wrong context, 4.9% abstention. Abstention trades correct responses to suppress wrong-context selections. The prototype does not enable abstention by default, as the preferred balance between wrong context and generic canned responses is application-specific.

J. Security Experiments

We evaluated router behavior under simulated GPS spoofing and BLE beacon cloning. When forged GPS fixes reported plausible coordinates (at 5 m accuracy) without the Android mock location flag, wrong-context selection rose to 38.9% (Table III). When Android’s mock location flag was detected, the router rejected the fix and fell back to valid sensors, reducing wrong selections to 1.7%. Cloned BLE beacons produced a 9.9% wrong-context rate, as BLE advertisements are unauthenticated.

K. Resource Overhead and Unmeasured Quantities

Physical energy consumption and battery drain attributable to the 60-second sensor sweep, the 15-minute background worker, and native priming were not measured with external power instrumentation. The memory footprint of 627 MB PSS (151 MB native heap, 391 MB memory-mapped model) represents a snapshot taken before configuring four sequence slots. Cache restore scaling across arbitrary token lengths was not measured.

VI. THREAT MODEL

The threat model considers two primary assets: the integrity of context selection (preventing adversarial manipulation of spoken utterances) and the confidentiality of serialized KV cache files (which encode prompt text).

We analyze three attacker profiles: *Attacker A* (software mock location), an unprivileged application on the same device advertising spoofed GPS coordinates through Android location APIs; *Attacker B* (physical radio impersonation), an adversary in physical proximity transmitting spoofed BLE advertisements matching registered beacon identifiers; and *Attacker C*

(storage extraction), an entity attempting to inspect or extract cached files from device storage or backups.

Security Boundaries and Mitigations. The router limits selection strictly to the catalog of five pre-authored contexts; sensor manipulation cannot inject arbitrary prompt text or alter user intent. Attacker A is mitigated by checking Android’s mock location flags (`isMock()` and `isFromMockProvider()`) and filtering anomalous accuracy readings; under active mock detection, wrong-context selection dropped to 1.7%. Against Attacker B, unauthenticated BLE beacons cannot be cryptographically verified; multi-modal scoring provides partial mitigation (9.9% wrong context under cloned beacons). Against Attacker C, KV files are isolated within application-private storage with backup disabled (`android:allowBackup="false"`), protecting cache contents unless the device operating system is rooted or physically compromised.

VII. LIMITATIONS AND CONCLUSION

This study has several limitations: (1) Benchmarks evaluate a single device (OnePlus 11R, Snapdragon 8+ Gen 1) running one quantized model (Qwen2.5-0.5B-Instruct); cross-device portability across different mobile chipsets remains unmeasured. (2) The prototype uses five pre-written contexts over synthetic behavioral traces; while the Markov baseline reaches 99.7% accuracy on this deterministic schedule, performance on irregular human mobility traces requires physical validation. (3) Trace simulations assume scheduled contexts are instantly available at the subsequent sweep, omitting runtime priming contention. (4) Background battery drain was not evaluated with a hardware power monitor. (5) The system was not tested with individuals who use AAC, and makes no clinical efficacy claims.

Within these constraints, the measurements demonstrate that proactive sensor-driven KV cache priming decouples context prompt prefill from user interaction on a smartphone. Across 30 randomized interleaved trials, CAP-KVC reduced matched 52-token prefill latency from 1847 ms to 366 ms (a $5.0\times$ reduction; Welch $t(32.3) = 15.5$, $p < 10^{-15}$), while state restoration added 2.9 ms. In trace simulations over 100 days, sensor-guided residency achieved 98.5% correct context selection versus 92.2% for request-driven LRU caching, trading cold misses on first visits for a 1.5% wrong-context rate. The primary vulnerability remains undetected GPS spoofing, which elevated wrong-context selections to 38.9%.

Source code, benchmark logs, and evaluation scripts are available at <https://github.com/omni-ar/AACBridge>.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to assist in drafting and editing sections of this manuscript, and to assist in writing and testing code across the Android application, JNI layer, trace evaluator, and evaluation scripts for Tables II–III and Figs. 2–3. Google Antigravity was used as an AI development assistant for Android/JNI components (Section IV). Fig. 1 was generated from TikZ code structured with Claude’s assistance

from author-specified layouts; Figs. 2–3 were plotted directly from empirical logs. No figures used generative image models. Research framing, system design, experiments, and interpretations remain the authors’.

REFERENCES

- [1] D. R. Beukelman and J. C. Light, *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 5th ed. Baltimore, MD, USA: Paul H. Brookes, 2020.
- [2] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [3] A. Yang *et al.*, “Qwen2.5 technical report,” *arXiv:2412.15115*, 2024.
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [5] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOS*, 2023.
- [8] I. Gim, G. Chen, S.-S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular attention reuse for low-latency inference,” in *Proc. MLSys*, 2024.
- [9] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, “RAG-Cache: Efficient knowledge caching for retrieval-augmented generation,” *ACM Trans. Comput. Syst.*, vol. 44, no. 1, Art. 2, Feb. 2026, doi: 10.1145/3768628.
- [10] J. Yao *et al.*, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in *Proc. EuroSys*, 2025.
- [11] Y. Liu *et al.*, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proc. ACM SIGCOMM*, 2024.
- [12] Z. Zhang *et al.*, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Proc. NeurIPS*, 2023.
- [13] Z. Liu *et al.*, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *Proc. NeurIPS*, 2023.
- [14] Y. Li *et al.*, “SnapKV: LLM knows what you are looking for before generation,” in *Proc. NeurIPS*, 2024.
- [15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *Proc. ICLR*, 2024.
- [16] C. Hooper *et al.*, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Proc. NeurIPS*, 2024.
- [17] Z. Liu *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” in *Proc. ICML*, 2024.
- [18] MLC Team, “MLC LLM,” 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [19] K. Alizadeh *et al.*, “LLM in a flash: Efficient large language model inference with limited memory,” in *Proc. ACL*, 2024.
- [20] Z. Xue *et al.*, “PowerInfer-2: Fast large language model inference on a smartphone,” *arXiv:2406.06282*, 2024.